

SUPPLY MANAGEMENT SYSTEM — Functional Specification Document

FSD Addendum 37 — Module Registry, Enablement Layer & Foundational Shared Entities

Property	Value
Document ID	SMS-FSD-ADD-037
Version	1.0
Date	2026-10-10
Status	Draft
Author	System Architect
Classification	Internal — Development
Depends On	SMS-FSD-ADD-030 (Manufacturing, Production Orders, BOM, Allocation Engine), SMS-FSD-ADD-031 (BOM Inline, Material Availability), SMS-FSD-ADD-036 (Service Orders & Service Fulfillment)
Architecture	.NET 8 / EF Core 8 / SQL Server 2022 / React 18 / Hangfire
Change Items	10 changes (C1-C10)
Impact	14 migrations, 8 new tables, 6 modified tables, 2 schema transfers, 1 new schema, 1 middleware, 1 DI service, 3 React context components, 1 admin UI panel

Table of Contents

- 1. [Purpose & Scope](#)
- 2. [C1 — Module Catalog & Dependencies](#)
- 3. [C2 — Module Features \(Sub-Module Feature Flags\)](#)
- 4. [C3 — Organization Module Licensing & Grace Period](#)
- 5. [C4 — Module Access Infrastructure \(Middleware, Attributes, Service\)](#)
- 6. [C5 — Product Capability Flags & Extension Table Pattern](#)
- 7. [C6 — BOM Schema Migration \(material → inventory\)](#)
- 8. [C7 — Fulfillment Routes & Product Routes](#)
- 9. [C8 — Customer Master \(New Schema\)](#)
- 10. [C9 — Cross-Module Awareness \(Workflow, Permissions, Sync Timestamps\)](#)
- 11. [C10 — Frontend Module Context & Module Administration UI](#)
- 12. [Database Migrations](#)
- 13. [API Endpoints](#)
- 14. [Permission Claims](#)
- 15. [UI Wireframes & Specifications](#)
- 16. [Business Rules](#)
- 17. [Test Scenarios](#)
- 18. [Development Phases](#)
- 19. [Future Enhancements](#)

1. Purpose & Scope

This addendum introduces the Module Registry and Enablement Layer — the foundational infrastructure that transforms SMS from a fixed-feature monolith into a configurable, module-toggleable SaaS product. After this addendum, any SMS module can be enabled or disabled per tenant without data loss, and individual features within a module can be toggled independently. This is the prerequisite for all future vertical modules (POS, Projects, CRM, Maintenance).

1.1 Design Principles

- **Schema Always Exists, Module Controls Visibility** — All database schemas and tables are always deployed for every tenant. Module enablement controls API access, UI visibility, and background job execution — never data existence. Disabling a module hides the feature; it never deletes data.
- **Data is Permanent, Access is the Switch** — Historical records created by a module remain fully visible in shared ledgers (Finance, Inventory) when that module is disabled. Only new record creation through the disabled module is blocked.
- **Extension Table Pattern for Module-Specific Fields** — Each module that extends a shared entity (e.g., Product) does so through a satellite table with a 1:1 FK, not by adding columns to the shared entity. This keeps the core entity clean and makes module-specific data naturally scoped.
- **Shared Entities Live in Shared Schemas** — Any entity consumed by more than one module lives in a base schema that is always on. BOM (used by both Production and Services) moves from `material` to `inventory`. Customers (used by future POS, Sales, CRM) gets its own always-on schema.
- **Feature Flags Within Modules** — A module may contain sub-features that can be individually toggled. A tenant might enable PRODUCTION but disable MRP Scheduling or Quality Inspection within it. This provides granular commercial licensing.
- **Grace Period on Disable** — When a module is disabled, in-progress records get a configurable grace period (default 30 days) to be completed, closed, or cancelled before they freeze.
- **Follow Odoo's Approach** — Module toggle controls menu visibility and API access. Database tables persist. Re-enabling a module restores full access to existing data instantly.

1.2 Changes Summary

Change ID	Title	Category
C1	Module Catalog & Dependencies	New tables — <code>tenant.Modules</code> , <code>tenant.ModuleDependencies</code> + seed data
C2	Module Features (Sub-Module Feature Flags)	New table — <code>tenant.ModuleFeatures</code> + seed data
C3	Organization Module Licensing & Grace Period	New tables — <code>tenant.OrgModules</code> , <code>tenant.OrgModuleFeatures</code> , <code>tenant.OrgModuleHistory</code>
C4	Module Access Infrastructure	New middleware, attribute, and DI service
C5	Product Capability Flags & Extension Table Pattern	Extend <code>inventory.Products</code> + new extension tables
C6	BOM Schema Migration	Transfer <code>BillOfMaterials</code> + <code>BomLines</code> from <code>material</code> to <code>inventory</code> schema
C7	Fulfillment Routes & Product Routes	New tables — <code>inventory.Routes</code> , <code>inventory.ProductRoutes</code>
C8	Customer Master	New schema <code>customers</code> — <code>customers.Customers</code> + seed data
C9	Cross-Module Awareness	Extend <code>workflow.ApprovalRules</code> , <code>auth.Permissions</code> + add <code>modified_at</code> timestamps
C10	Frontend Module Context & Module Administration UI	React ModuleProvider, ModuleGuard, Admin panel

1.3 Module Classification

Classification	Modules	Rule
Always-On (cannot be disabled)	CORE, INVENTORY, FINANCE, CUSTOMERS	These are skeletal — every business tenant needs them
Toggleable (per-tenant)	SUPPLIERS, PROCUREMENT, DEMAND, WAREHOUSE, LOGISTICS, PRODUCTION, SERVICES	Enable/disable based on business type and license tier
Future (placeholder catalog entries)	POS, PROJECTS, CRM, MAINTENANCE, ECOMMERCE	Registered in catalog but not yet implemented

1.4 Module Dependency Graph



2. C1 — Module Catalog & Dependencies

2.1 Rationale

A central registry of all modules — existing, toggleable, and future — with declared dependencies between them. This is the master catalog that the licensing system, middleware, UI, and background jobs all reference.

2.2 Entities

2.2.1 `tenant.Modules`

Column	Type	Constraints	Description
module_id	INT	PK, IDENTITY	Auto-increment PK
code	VARCHAR(30)	NOT NULL, UNIQUE	Module code: CORE , INVENTORY , PRODUCTION , POS , etc.
name	NVARCHAR(100)	NOT NULL	Display name: "Core", "Inventory", "Production", "Point of Sale"
description	NVARCHAR(500)	NULL	Short description of module capabilities
schema_names	VARCHAR(200)	NOT NULL	Comma-separated schema names this module owns: auth , tenant , lookups , workflow , hangfire , reports for CORE
is_always_on	BIT	NOT NULL, DEFAULT 0	If true, module cannot be disabled by any tenant
is_available	BIT	NOT NULL, DEFAULT 1	If false, module is a future placeholder — not yet activatable
icon	VARCHAR(50)	NULL	Icon identifier for UI display
sort_order	INT	NOT NULL, DEFAULT 0	Display order in admin panel
created_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	Row creation timestamp

Indexes:

- PK: module_id
- UQ: code

2.2.2 tenant.ModuleDependencies

Column	Type	Constraints	Description
module_id	INT	NOT NULL, FK → Modules	The dependent module
depends_on_module_id	INT	NOT NULL, FK → Modules	The required module

Constraints:

- PK: (module_id, depends_on_module_id)
- CHECK: module_id <> depends_on_module_id — no self-dependency
- No circular dependencies enforced at application level

2.3 Seed Data — Module Catalog

code	name	schema_names	is_always_on	is_available	depends_on
CORE	Core Platform	auth,tenant,lookups,workflow,hangfire,reports	1	1	—
INVENTORY	Inventory Management	inventory	1	1	—
FINANCE	Finance & Accounting	finance	1	1	—
CUSTOMERS	Customer Management	customers	1	1	—
SUPPLIERS	Supplier Management	suppliers	0	1	—
PROCUREMENT	Procurement	procurement	0	1	INVENTORY, SUPPLIERS
DEMAND	Demand Planning	demand	0	1	PROCUREMENT
WAREHOUSE	Warehouse Operations	warehouse	0	1	INVENTORY
LOGISTICS	Logistics & Shipping	logistics	0	1	WAREHOUSE
PRODUCTION	Manufacturing & Production	material	0	1	INVENTORY
SERVICES	Service Management	service	0	1	INVENTORY
POS	Point of Sale	pos	0	0	INVENTORY, FINANCE, CUSTOMERS
PROJECTS	Project Management	projects	0	0	INVENTORY
CRM	Customer Relationship	crm	0	0	CUSTOMERS
MAINTENANCE	Maintenance / CMMS	maintenance	0	0	INVENTORY
ECOMMERCE	E-Commerce Integration	ecommerce	0	0	INVENTORY, CUSTOMERS

Note on `material` schema: The PRODUCTION module owns the `material` schema. However, after C6 (BOM Migration), `BillOfMaterials` and `BomLines` move to the `inventory` schema and become shared entities accessible regardless of PRODUCTION enablement.

3. C2 — Module Features (Sub-Module Feature Flags)

3.1 Rationale

Within a module, not all features need to be active simultaneously. A tenant might enable PRODUCTION but not need MRP Scheduling or Quality Inspection yet. Feature flags provide granular control below the module level, enabling finer-grained commercial licensing (e.g., "Production Basic" vs. "Production Advanced").

3.2 Entity

3.2.1 tenant.ModuleFeatures

Column	Type	Constraints	Description
feature_id	INT	PK, IDENTITY	Auto-increment PK
module_id	INT	NOT NULL, FK → Modules	Parent module
code	VARCHAR(50)	NOT NULL	Feature code: MRP_SCHEDULING, QUALITY_INSPECTION, etc.
name	NVARCHAR(100)	NOT NULL	Display name
description	NVARCHAR(500)	NULL	What this feature provides
is_core_feature	BIT	NOT NULL, DEFAULT 0	If true, always on when the parent module is enabled — cannot be individually disabled
is_available	BIT	NOT NULL, DEFAULT 1	If false, feature is planned but not yet implemented
requires_feature_id	INT	NULL, FK → self	Another feature in the same module that must be on first
sort_order	INT	NOT NULL, DEFAULT 0	Display order within module

Constraints:

- UQ: (module_id, code)

3.3 Seed Data — Features per Module

INVENTORY Module

code	name	is_core_feature	is_available
PRODUCT_CATALOG	Product Catalog	1 (core)	1
STOCK_LEVELS	Stock Level Tracking	1 (core)	1
STOCK_ADJUSTMENTS	Stock Adjustments	1 (core)	1
STOCK_TRANSFERS	Inter-Warehouse Transfers	0	1
STOCK_COUNTS	Stock Counting / Cycle Count	0	1
PRODUCT_VARIANTS	Product Variants (Size, Color)	0	0
BOM_MANAGEMENT	Bill of Materials	0	1

BOM_MANAGEMENT is in INVENTORY, not PRODUCTION. After C6, BOM is a shared entity in the **inventory** schema. The BOM_MANAGEMENT feature flag controls BOM tab visibility on the Product form. It is auto-enabled when either PRODUCTION or SERVICES module is enabled (see Business Rule MOD-08).

SUPPLIERS Module

code	name	is_core_feature	is_available
SUPPLIER_MASTER	Supplier Master Data	1 (core)	1
SUPPLIER_CONTACTS	Supplier Contacts	1 (core)	1
SUPPLIER_EVALUATION	Supplier Rating & Evaluation	0	1

PROCUREMENT Module

code	name	is_core_feature	is_available
PURCHASE_ORDERS	Purchase Orders	1 (core)	1
GRN_MANAGEMENT	Goods Receipt Notes	1 (core)	1
RFQ_MANAGEMENT	Request for Quotation	0	1
PURCHASE_RETURNS	Purchase Returns	0	1
BLANKET_ORDERS	Blanket / Standing Orders	0	0

DEMAND Module

code	name	is_core_feature	is_available
DEMAND_FORECASTING	Demand Forecasting	1 (core)	1
REORDER_SUGGESTIONS	Reorder Point Suggestions	1 (core)	1

WAREHOUSE Module

code	name	is_core_feature	is_available
BIN_MANAGEMENT	Bin / Location Management	1 (core)	1
PICK_LISTS	Pick List Generation	0	1
PUT_AWAY_RULES	Put-Away Rules	0	1
CYCLE_COUNTS	Cycle Count Scheduling	0	1
BARCODE_SCANNING	Barcode / QR Scanning	0	0

LOGISTICS Module

code	name	is_core_feature	is_available
DELIVERY_ORDERS	Delivery Orders	1 (core)	1
SHIPMENT_TRACKING	Shipment Tracking	0	1
CARRIER_INTEGRATION	Carrier API Integration	0	0
RETURN_ORDERS	Return Orders	0	1

PRODUCTION Module

code	name	is_core_feature	is_available
PRODUCTION_ORDERS	Production Orders	1 (core)	1
MATERIAL_REQUIREMENTS	Material Requirement Planning	1 (core)	1
MRP_SCHEDULING	Automated MRP Scheduling	0	1
QUALITY_INSPECTION	Quality Inspection (Incoming/In-process)	0	1
WORK_CENTERS	Work Center Management	0	0
PRODUCTION_COSTING	Production Cost Analysis	0	1

SERVICES Module

code	name	is_core_feature	is_available
SERVICE_ORDERS	Service Orders	1 (core)	1
SERVICE_BOM	Service BOM & Material Consumption	1 (core)	1
FIELD_SERVICE	Field Service Management	0	0
SERVICE_CONTRACTS	Service Contracts / AMC	0	0

FINANCE Module

code	name	is_core_feature	is_available
CHART_OF_ACCOUNTS	Chart of Accounts	1 (core)	1
JOURNAL_ENTRIES	Journal Entries	1 (core)	1
BANK_RECONCILIATION	Bank Reconciliation	0	0
MULTI_CURRENCY	Multi-Currency Support	0	1
BUDGETING	Budget Management	0	0

CUSTOMERS Module

code	name	is_core_feature	is_available
CUSTOMER_MASTER	Customer Master Data	1 (core)	1
CREDIT_MANAGEMENT	Customer Credit Limits	0	1
LOYALTY_PROGRAM	Customer Loyalty Points	0	0

4. C3 — Organization Module Licensing & Grace Period

4.1 Rationale

Each tenant independently controls which modules and features are active. When a module is disabled, a grace period allows in-progress records to be completed before they freeze. Full audit trail tracks all enable/disable actions.

4.2 Entities

4.2.1 tenant.OrgModules

Column	Type	Constraints	Description
org_module_id	BIGINT	PK, IDENTITY	Auto-increment PK
org_id	BIGINT	NOT NULL, FK → Organizations	Tenant
module_id	INT	NOT NULL, FK → Modules	Module
is_enabled	BIT	NOT NULL, DEFAULT 1	Current enablement status
license_tier	TINYINT	NOT NULL, DEFAULT 0	0=Starter, 1=Professional, 2=Enterprise
enabled_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	When first enabled
enabled_by	BIGINT	NOT NULL, FK → Users	Who enabled
disabled_at	DATETIME2(2)	NULL	When disabled (NULL if currently enabled)
disabled_by	BIGINT	NULL, FK → Users	Who disabled
grace_period_ends_at	DATETIME2(2)	NULL	Grace period expiry (NULL if enabled or grace ended)
trial_expires_at	DATETIME2(2)	NULL	Trial period expiry (NULL for full license)

Constraints:

- UQ: (org_id, module_id) — one record per org per module
- CHECK: grace_period_ends_at IS NULL OR disabled_at IS NOT NULL — grace only applies when disabled

4.2.2 tenant.OrgModuleFeatures

Column	Type	Constraints	Description
org_module_feature_id	BIGINT	PK, IDENTITY	Auto-increment PK
org_id	BIGINT	NOT NULL, FK → Organizations	Tenant
feature_id	INT	NOT NULL, FK → ModuleFeatures	Feature
is_enabled	BIT	NOT NULL, DEFAULT 1	Feature enablement status
enabled_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	When enabled
enabled_by	BIGINT	NOT NULL, FK → Users	Who enabled

Constraints:

- UQ: (org_id, feature_id)
- Feature's parent module must be enabled in OrgModules (enforced at application level)

4.2.3 tenant.OrgModuleHistory

Column	Type	Constraints	Description
history_id	BIGINT	PK, IDENTITY	Auto-increment PK
org_id	BIGINT	NOT NULL, FK → Organizations	Tenant
module_id	INT	NOT NULL, FK → Modules	Module
feature_id	INT	NULL, FK → ModuleFeatures	Feature (NULL if module-level action)
action	TINYINT	NOT NULL	0=Enabled, 1=Disabled, 2=FeatureEnabled, 3=FeatureDisabled, 4=TierChanged, 5=GracePeriodExpired
performed_by	BIGINT	NOT NULL, FK → Users	Who performed the action
performed_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	When performed
previous_tier	TINYINT	NULL	Previous license tier (for TierChanged action)
new_tier	TINYINT	NULL	New license tier
notes	NVARCHAR(500)	NULL	Optional notes

Indexes:

- IX: (org_id, module_id, performed_at DESC) — history lookup

4.3 Grace Period Behavior

When a module is disabled:

Phase	Duration	Create New	Update Existing	Read Historical	Background Jobs
Grace Period	30 days (configurable per org)	❌ Blocked	✅ Status transitions only (complete, close, cancel)	✅ Full access	❌ Skipped
After Grace	Indefinite until re-enabled	❌ Blocked	❌ Frozen	✅ Full access	❌ Skipped
Re-enabled	Immediate	✅ Full access	✅ Full access	✅ Full access	✅ Resume

4.4 Initial Seed — All Existing Tenants

On migration, every existing tenant gets all currently-implemented modules enabled:

```
sql

INSERT INTO tenant.OrgModules (org_id, module_id, is_enabled, license_tier, enabled_at, enabled_by)
SELECT o.org_id, m.module_id, 1, 0, SYSUTCDATETIME(),
       (SELECT TOP 1 user_id FROM auth.Users WHERE org_id = o.org_id AND is_admin = 1)
FROM tenant.Organizations o
CROSS JOIN tenant.Modules m
WHERE m.is_available = 1;
```

5. C4 — Module Access Infrastructure

5.1 Rationale

A centralized access layer that gates every API request, background job, and UI component based on module and feature enablement. This is the enforcement mechanism for the Module Registry.

5.2 Components

5.2.1 IModuleService Interface

```
csharp
```

```

public interface IModuleService
{
    /// <summary>Check if module is enabled for the current tenant</summary>
    bool IsModuleEnabled(string moduleCode);

    /// <summary>Check if module is enabled for a specific tenant</summary>
    bool IsModuleEnabled(long orgId, string moduleCode);

    /// <summary>Check if feature is enabled (implies parent module is enabled)</summary>
    bool IsFeatureEnabled(string featureCode);

    /// <summary>Check if currently in grace period for a disabled module</summary>
    bool IsInGracePeriod(string moduleCode);

    /// <summary>Get all enabled module codes for current tenant</summary>
    Task<List<string>> GetEnabledModulesAsync();

    /// <summary>Get all enabled feature codes for current tenant</summary>
    Task<List<string>> GetEnabledFeaturesAsync();

    /// <summary>Enable a module (validates dependencies)</summary>
    Task<ModuleActionResult> EnableModuleAsync(string moduleCode, int licenseTier = 0);

    /// <summary>Disable a module (validates dependents, sets grace period)</summary>
    Task<ModuleActionResult> DisableModuleAsync(string moduleCode, int graceDays = 30);

    /// <summary>Enable/disable a feature within an enabled module</summary>
    Task<ModuleActionResult> SetFeatureEnabledAsync(string featureCode, bool enabled);
}

```

Implementation Notes:

- `ModuleService` caches enabled modules per-request using `IHttpContextAccessor` — one DB hit per request, not per check.
- Uses `IMemoryCache` with a 5-minute sliding expiration for cross-request caching, invalidated on enable/disable actions.
- Registered as **Scoped** in DI (one instance per request).

5.2.2 `[RequiresModule]` Attribute

csharp

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method, AllowMultiple = false)]
public class RequiresModuleAttribute : Attribute
{
    public string ModuleCode { get; }
    public bool AllowReadWhenDisabled { get; set; } = false;
    public bool AllowDuringGracePeriod { get; set; } = true;

    public RequiresModuleAttribute(string moduleCode)
    {
        ModuleCode = moduleCode;
    }
}
```

Property	Default	Description
ModuleCode	(required)	Module code to check
AllowReadWhenDisabled	false	If true, GET requests pass even when module is disabled (for viewing historical data)
AllowDuringGracePeriod	true	If true, state-transition PUT/PATCH requests pass during grace period

5.2.3 **[RequiresFeature]** Attribute

```
csharp
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method, AllowMultiple = false)]
public class RequiresFeatureAttribute : Attribute
{
    public string FeatureCode { get; }

    public RequiresFeatureAttribute(string featureCode)
    {
        FeatureCode = featureCode;
    }
}
```

5.2.4 **ModuleAccessMiddleware**

Runs in the ASP.NET Core pipeline **after** authentication and tenant resolution, **before** controller action execution:

```
Request → Authentication → Tenant Resolution → ModuleAccessMiddleware → Controller
```

Logic:

1. Read **[RequiresModule]** from the target controller/action
2. If no attribute → pass through (CORE endpoints)
3. If module is enabled → pass through
4. If module is disabled and **AllowReadWhenDisabled = true** and method is GET → pass through
5. If module is disabled and in grace period and **AllowDuringGracePeriod = true** and method is PUT/PATCH → pass through (state transitions only)
6. Otherwise → return **403 Forbidden** with body:

```
json
{
  "error": "MODULE_NOT_LICENSED",
  "module": "PRODUCTION",
  "message": "The Production module is not enabled for your organization.",
  "graceEndsAt": null
}
```

If in grace period, include `graceEndsAt` in the response.

5.2.5 Existing Controller Retrofit

Every existing controller gets the `[RequiresModule]` attribute. **No controller logic changes — only the attribute is added:**

Module Code	Controllers to Tag
CORE	(no tag needed — CORE is always on, middleware skips untagged controllers)
INVENTORY	ProductsController, StockLevelsController, StockMovementsController, StockAdjustmentsController, BomController (after C6 migration)
SUPPLIERS	SuppliersController, SupplierContactsController
PROCUREMENT	PurchaseOrdersController, PurchaseOrderLinesController, GrnController
DEMAND	DemandForecastsController, ReorderSuggestionsController
WAREHOUSE	WarehousesController, BinsController, PickListsController, PutAwayController
LOGISTICS	DeliveryOrdersController, ShipmentsController, CarriersController
PRODUCTION	ProductionOrdersController, MaterialRequirementsController, MaterialIssuesController, FGReceiptController, ProductionLedgerController
SERVICES	ServiceOrdersController, ServiceMaterialRequirementsController, ServiceLedgerController
FINANCE	JournalEntriesController, ChartOfAccountsController, LedgerController
CUSTOMERS	CustomersController (new in C8)

MaterialIssuesController special case: Material Issues are shared by Production and Services (Addendum 36 §C6 — polymorphic extension). The controller needs dual-module logic: if the Material Issue has `production_order_id`, check PRODUCTION; if it has `service_order_id`, check SERVICES. This is handled at the action level with `[RequiresModule]` on individual methods, not the controller class.

5.2.6 Background Job Module Check

A base class for all Hangfire recurring jobs that process per-tenant:

```
csharp
```

```

public abstract class ModuleAwareJob
{
    protected readonly IModuleService _moduleService;

    protected abstract string RequiredModuleCode { get; }

    protected async Task ProcessAllTenants(Func<long, Task> processFunc)
    {
        var tenants = await _tenantService.GetAllActiveAsync();
        foreach (var tenant in tenants)
        {
            if (!_moduleService.IsModuleEnabled(tenant.OrgId, RequiredModuleCode))
                continue;

            await processFunc(tenant.OrgId);
        }
    }
}

```

Existing jobs to retrofit:

Job	Module Check
MRP Calculation Job	PRODUCTION
Demand Forecast Refresh	DEMAND
Reorder Point Check	DEMAND
Allocation Engine Batch	INVENTORY (always runs)
Delivery Order Status Sync	LOGISTICS
Hangfire Dashboard	CORE (always runs)

6. C5 — Product Capability Flags & Extension Table Pattern

6.1 Rationale

Products are shared across all modules. Rather than module toggles controlling which products appear where, **product capability flags** indicate what operations a product supports. Module-specific fields live in extension tables (satellite pattern) — not on the core Products table.

6.2 Schema Changes to `inventory.Products`

New columns added to existing `inventory.Products` table:

Column	Type	Constraints	Description
<code>is_purchasable</code>	BIT	NOT NULL, DEFAULT 1	Can appear in Purchase Order lines
<code>is_sellable</code>	BIT	NOT NULL, DEFAULT 0	Can appear in Sale Order lines, POS product grid
<code>is_manufacturable</code>	BIT	NOT NULL, DEFAULT 0	Can be the output of a Production Order (has/can have BOM)
<code>is_serviceable</code>	BIT	NOT NULL, DEFAULT 0	Can be delivered via a Service Order
<code>is_stockable</code>	BIT	NOT NULL, DEFAULT 1	Tracks physical inventory (false for pure service items)
<code>modified_at</code>	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	Last modification timestamp for delta sync

`is_purchasable` + `is_sellable` control which screens show this product. These flags are independent of module enablement. A product marked `is_sellable = false` never appears in a Sales Order lookup even if the Sales module is enabled. A product marked `is_manufacturable = true` retains that flag even when Production is disabled — the flag is dormant, not removed.

6.3 Extension Table Pattern

Each module that needs product-specific fields creates a 1:1 extension table in its own schema:

```
inventory.Products (core – always exists)
├── 1:1 → material.ProductProductionSettings (PRODUCTION extension)
├── 1:1 → service.ProductServiceSettings (SERVICES extension)
├── 1:1 → pos.ProductPosSettings (POS extension – future, defined in POS addendum)
└── 1:1 → (future module extensions follow same pattern)
```

6.3.1 `material.ProductProductionSettings`

Column	Type	Constraints	Description
<code>product_id</code>	BIGINT	PK, FK → <code>inventory.Products</code>	1:1 with product
<code>default_bom_id</code>	BIGINT	NULL, FK → <code>inventory.BillOfMaterials</code>	Default BOM for this product
<code>production_type</code>	TINYINT	NOT NULL, DEFAULT 0	0=MakeToStock, 1=MakeToOrder, 2=Assemble
<code>scrap_percentage</code>	DECIMAL(5,2)	NOT NULL, DEFAULT 0	Expected scrap rate
<code>manufacturing_lead_time_days</code>	INT	NOT NULL, DEFAULT 1	Lead time for production planning
<code>default_work_center_id</code>	BIGINT	NULL	Default work center (future — WORK_CENTERS feature)

Migration Note: If any production-specific fields currently exist on `inventory.Products`, this migration extracts them into this table. Otherwise, this is a net-new table.

6.3.2 service.ProductServiceSettings

Column	Type	Constraints	Description
product_id	BIGINT	PK, FK → inventory.Products	1:1 with product
default_service_bom_id	BIGINT	NULL, FK → inventory.BillOfMaterials	Default service BOM
service_category	TINYINT	NOT NULL, DEFAULT 0	0=General, 1=Installation, 2=Repair, 3=Maintenance, 4=Consulting
estimated_duration_hours	DECIMAL(6,2)	NULL	Estimated service duration
requires_site_visit	BIT	NOT NULL, DEFAULT 0	Requires on-site service delivery
is_subcontractable	BIT	NOT NULL, DEFAULT 0	Can be fulfilled by external vendor

6.4 Product API — Conditional Extension Loading

The Product GET endpoint conditionally includes extension data based on module enablement:

```
GET /api/products/{id}
```

Response when PRODUCTION + SERVICES enabled:

```
json
{
  "productId": 1,
  "name": "HVAC Installation Kit",
  "sku": "HVAC-KIT-001",
  "productType": 6,
  "isPurchasable": true,
  "isSellable": true,
  "isManufacturable": false,
  "isServiceable": true,
  "isStockable": true,
  "productionSettings": null,
  "serviceSettings": {
    "defaultServiceBomId": 47,
    "serviceCategory": 1,
    "estimatedDurationHours": 4.00,
    "requiresSiteVisit": true,
    "isSubcontractable": false
  }
}
```

Same product when only PRODUCTION enabled (SERVICES disabled):

```
json
```

```
{
  "productId": 1,
  "name": "HVAC Installation Kit",
  "...": "...",
  "productionSettings": null
  // serviceSettings key absent entirely
}
```

EF Core query pattern:

```
csharp

var query = _db.Products.AsQueryable();

if (_moduleService.IsModuleEnabled("PRODUCTION"))
    query = query.Include(p => p.ProductionSettings);
if (_moduleService.IsModuleEnabled("SERVICES"))
    query = query.Include(p => p.ServiceSettings);
// Future: if (_moduleService.IsModuleEnabled("POS"))
//     query = query.Include(p => p.PosSettings);
```

7. C6 — BOM Schema Migration (material → inventory)

7.1 Rationale

BOM is consumed by both Production Orders (Addendum 30) and Service Orders (Addendum 36). Currently, `BillOfMaterials` and `BomLines` reside in the `material` schema, which is owned by the PRODUCTION module. If a tenant disables PRODUCTION but keeps SERVICES enabled, they lose access to BOM — breaking Service Orders that depend on BOM explosion.

Solution: Move BOM tables to the `inventory` schema (always on). BOM becomes a shared product-structure entity accessible by any module. The BOM_MANAGEMENT feature flag in the INVENTORY module controls UI visibility (auto-enabled when PRODUCTION or SERVICES is active).

7.2 Tables Transferred

Current Location	New Location	Entity
<code>material.BillOfMaterials</code>	<code>inventory.BillOfMaterials</code>	BOM header
<code>material.BomLines</code>	<code>inventory.BomLines</code>	BOM line items

7.3 Schema Transfer Migration

```
sql
```

```
-- M037.6: Transfer BOM tables from material to inventory schema

-- Step 1: Transfer tables
ALTER SCHEMA inventory TRANSFER material.BillOfMaterials;
ALTER SCHEMA inventory TRANSFER material.BomLines;

-- Step 2: Update FK references in material schema
-- ProductionOrders.bom_id already references BillOfMaterials by ID –
-- the FK constraint follows the table to its new schema automatically in SQL Server.

-- Step 3: Update FK references in service schema (Addendum 36)
-- ServiceOrders.bom_id references BillOfMaterials by ID – same automatic follow.

-- Step 4: If any stored procedures, views, or computed columns reference
-- material.BillOfMaterials, update them to inventory.BillOfMaterials.

-- Step 5: Verify index names (optional rename for consistency)
-- Indexes follow the table transfer automatically.
```

SQL Server `ALTER SCHEMA TRANSFER` behavior: When a table is transferred to a new schema, all FK constraints, indexes, and triggers follow automatically. No data movement occurs — it's a metadata change. FK constraints from other tables pointing to the transferred table remain valid.

7.4 Impact on Existing Code

Component	Change Required
EF Core <code>MaterialDbContext</code>	Remove <code>BillOfMaterials</code> and <code>BomLines</code> DbSet declarations
EF Core <code>InventoryDbContext</code>	Add <code>BillOfMaterials</code> and <code>BomLines</code> DbSet declarations with <code>inventory</code> schema mapping
<code>IBomService</code>	Interface stays the same; implementation moves from Material module to Inventory module DI registration
<code>ProductionBomExplosionService</code>	No change — injects <code>IBomService</code> (now from Inventory module)
<code>ServiceBomExplosionService</code>	No change — injects <code>IBomService</code> (now from Inventory module)
<code>BomController</code>	Moves from <code>[RequiresModule("PRODUCTION")]</code> to <code>[RequiresModule("INVENTORY")]</code>
Product form BOM tab	Visibility controlled by <code>BOM_MANAGEMENT</code> feature flag, not PRODUCTION module
API route	Changes from <code>/api/production/bom</code> to <code>/api/bom</code> (top-level, module-neutral)

7.5 BOM Sharing Rules

Scenario	BOM Accessible?	Can Create New BOM?	BOM Tab Visible?
PRODUCTION + SERVICES	Yes	Yes	Yes
PRODUCTION + SERVICES	Yes	Yes	Yes
PRODUCTION + SERVICES	Yes	Yes	Yes
PRODUCTION + SERVICES	Data exists	No (BOM_MANAGEMENT auto-disabled)	Hidden
BOM_MANAGEMENT manually enabled (no PROD/SVC)	Yes	Yes	Yes

BOM_MANAGEMENT auto-enable rule (MOD-08): When PRODUCTION or SERVICES is enabled, the system automatically enables the BOM_MANAGEMENT feature in the INVENTORY module. When both are disabled, BOM_MANAGEMENT is auto-disabled unless it was explicitly enabled by the admin (tracked via `enabled_by` — system auto-enable uses a system user ID, manual enable uses the admin's user ID; auto-disable only reverses system-enabled entries).

7.6 BOM Usage Classification

New column on `inventory.BillOfMaterials`:

Column	Type	Constraints	Description
<code>bom_usage</code>	TINYINT	NOT NULL, DEFAULT 0	0=Universal, 1=ProductionPreferred, 2=ServicePreferred, 3=PosKit (future)

This is a **business classification**, not a licensing gate. A Universal BOM can be used by any module. A ProductionPreferred BOM appears first in Production Order BOM selection. A ServicePreferred BOM appears first in Service Order BOM selection. No BOM is hidden based on this field — it's a sort/filter preference.

8. C7 — Fulfillment Routes & Product Routes

8.1 Rationale

Routes define how demand for a product is fulfilled. The "Manufacture" route requires the PRODUCTION module; "Service Fulfillment" requires SERVICES; "Pick-Pack-Ship" requires WAREHOUSE. When a module is disabled, its routes become unavailable — dynamically, without modifying product configuration.

8.2 Entities

8.2.1 `inventory.Routes`

Column	Type	Constraints	Description
route_id	INT	PK, IDENTITY	Auto-increment PK
org_id	BIGINT	NOT NULL, FK → Organizations	Tenant
code	VARCHAR(30)	NOT NULL	Route code: BUY, MANUFACTURE, SERVICE, PPS, DIRECT_SHIP
name	NVARCHAR(100)	NOT NULL	Display name
description	NVARCHAR(300)	NULL	What this route does
requires_module	VARCHAR(30)	NULL	Module code required for this route (NULL = always available)
requires_feature	VARCHAR(50)	NULL	Feature code required (NULL = any feature within module)
sequence	INT	NOT NULL, DEFAULT 0	Priority — lower number = higher priority
is_active	BIT	NOT NULL, DEFAULT 1	Admin can manually deactivate
created_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
modified_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	

Constraints:

- UQ: (org_id, code)

8.2.2 inventory.ProductRoutes

Column	Type	Constraints	Description
product_id	BIGINT	NOT NULL, FK → Products	Product
route_id	INT	NOT NULL, FK → Routes	Route
is_preferred	BIT	NOT NULL, DEFAULT 0	Default route when multiple are available
org_id	BIGINT	NOT NULL, FK → Organizations	Tenant (denormalized for query filter)

Constraints:

- PK: (product_id, route_id)
- Only one is_preferred = 1 per product per org (enforced at application level)

8.3 Seed Data — Default Routes per Org

```
sql
```

```
INSERT INTO inventory.Routes (org_id, code, name, requires_module, sequence)
SELECT org_id, code, name, requires_module, seq
FROM tenant.Organizations
CROSS APPLY (VALUES
    ('BUY', 'Purchase (Buy)', 'PROCUREMENT', 10),
    ('MANUFACTURE', 'Manufacture', 'PRODUCTION', 20),
    ('SERVICE', 'Service Fulfillment', 'SERVICES', 30),
    ('MTO', 'Make to Order', 'PRODUCTION', 40),
    ('PPS', 'Pick → Pack → Ship', 'WAREHOUSE', 50),
    ('DIRECT_SHIP', 'Direct Ship (Drop-Ship)', 'PROCUREMENT', 60),
    ('INTERNAL', 'Internal Transfer', NULL, 70)
) AS r(code, name, requires_module, seq);
```

8.4 Available Routes Query

```
csharp
public async Task<List<Route>> GetAvailableRoutesAsync(long productId)
{
    return await _db.ProductRoutes
        .Where(pr => pr.ProductId == productId)
        .Include(pr => pr.Route)
        .Where(pr => pr.Route.IsActive)
        .Where(pr => pr.Route.RequiresModule == null
            || _moduleService.IsModuleEnabled(pr.Route.RequiresModule))
        .Where(pr => pr.Route.RequiresFeature == null
            || _moduleService.IsFeatureEnabled(pr.Route.RequiresFeature))
        .OrderBy(pr => pr.Route.Sequence)
        .Select(pr => pr.Route)
        .ToListAsync();
}
```

Behavior when modules change:

Product Routes Config	All Modules On	Production Off	Warehouse Off	Only Procurement On
BUY + MANUFACTURE + PPS	BUY, MANUFACTURE, PPS	BUY, PPS	BUY, MANUFACTURE	BUY
MANUFACTURE + MTO	MANUFACTURE, MTO	(none available)	MANUFACTURE, MTO	(none available)
SERVICE + BUY	SERVICE, BUY	SERVICE, BUY	SERVICE, BUY	BUY

9. C8 — Customer Master (New Schema)

9.1 Rationale

POS, Sales, CRM, and Projects all need a customer entity. Like Suppliers for the purchasing side, Customers is a shared entity in its own always-on schema. This addendum creates the foundation; POS and Sales addendums extend it.

9.2 New Schema

```
sql
CREATE SCHEMA customers;
```

9.3 Entity

9.3.1 customers.Customers

Column	Type	Constraints	Description
customer_id	BIGINT	PK, IDENTITY	Auto-increment PK
org_id	BIGINT	NOT NULL, FK → Organizations	Tenant
code	VARCHAR(20)	NOT NULL	Auto-generated customer code (C-00001)
name	NVARCHAR(200)	NOT NULL	Customer name (individual or company)
customer_type	TINYINT	NOT NULL, DEFAULT 1	0=WalkIn, 1=Individual, 2=Company, 3=Employee
email	NVARCHAR(200)	NULL	Primary email
phone	VARCHAR(30)	NULL	Primary phone
mobile	VARCHAR(30)	NULL	Mobile number
tax_id	VARCHAR(50)	NULL	Tax registration number (for B2B invoicing)
credit_limit	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Maximum outstanding receivable
current_balance	DECIMAL(18,4)	NOT NULL, DEFAULT 0	Current outstanding receivable
loyalty_points	INT	NOT NULL, DEFAULT 0	Loyalty points balance (future)
payment_terms_days	INT	NOT NULL, DEFAULT 0	0 = immediate, 30 = net-30, etc.
currency_code	VARCHAR(3)	NOT NULL, DEFAULT 'USD'	Default currency
address_line1	NVARCHAR(200)	NULL	
address_line2	NVARCHAR(200)	NULL	
city	NVARCHAR(100)	NULL	
state_province	NVARCHAR(100)	NULL	
postal_code	VARCHAR(20)	NULL	
country_code	VARCHAR(3)	NULL	ISO 3166-1 alpha-3
notes	NVARCHAR(MAX)	NULL	Internal notes
is_active	BIT	NOT NULL, DEFAULT 1	
created_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	
created_by	BIGINT	NOT NULL, FK → Users	
modified_at	DATETIME2(2)	NOT NULL, DEFAULT SYSUTCDATETIME()	For delta sync
modified_by	BIGINT	NULL, FK → Users	

Column	Type	Constraints	Description
row_version	ROWVERSION	NOT NULL	Optimistic concurrency

Indexes:

- UQ: (org_id, code) — unique customer code per tenant
- IX: (org_id, name) — name search
- IX: (org_id, email) — email lookup
- IX: (org_id, phone) — phone lookup
- IX: (org_id, customer_type) — type filter
- IX: (org_id, modified_at) — delta sync

EF Core Query Filter:

```
csharp
builder.HasQueryFilter(c => c.OrgId == _currentTenant.OrgId);
```

9.4 Seed Data — Walk-In Customer

Every org gets a default Walk-In Customer for anonymous transactions (especially POS):

```
sql
INSERT INTO customers.Customers (org_id, code, name, customer_type, is_active, created_by)
SELECT org_id, 'C-WALKIN', 'Walk-In Customer', 0, 1,
       (SELECT TOP 1 user_id FROM auth.Users WHERE org_id = o.org_id AND is_admin = 1)
FROM tenant.Organizations o;
```

9.5 Customer API

#	Method	Endpoint	Description
1	GET	/api/customers	List customers (paginated, filterable by type, search by name/phone/email)
2	GET	/api/customers/{id}	Get customer detail
3	POST	/api/customers	Create customer
4	PUT	/api/customers/{id}	Update customer
5	PATCH	/api/customers/{id}/status	Activate/deactivate
6	GET	/api/customers/search?q={term}	Quick search (name, phone, email) for POS/SO lookups
7	GET	/api/customers/{id}/balance	Current outstanding balance

Controller:

```
csharp
```

```
[ApiController]
[Route("api/customers")]
[RequiresModule("CUSTOMERS")] // Always on – but tagged for consistency
public class CustomersController : ControllerBase { }
```

10. C9 — Cross-Module Awareness (Workflow, Permissions, Sync Timestamps)

10.1 Rationale

Existing cross-cutting entities (approval rules, permissions) need module association so they behave correctly when modules are toggled. Additionally, shared entities need `modified_at` timestamps for future delta-sync support (POS, mobile, offline-first clients).

10.2 Workflow Rules — Module Association

New column on `workflow.ApprovalRules`:

Column	Type	Constraints	Description
<code>module_code</code>	VARCHAR(30)	NULL	Module this rule belongs to (NULL = applies globally)

Backfill existing rules:

```
sql
UPDATE workflow.ApprovalRules SET module_code = 'PRODUCTION' WHERE entity_type IN ('ProductionOrder', 'MaterialOrder');
UPDATE workflow.ApprovalRules SET module_code = 'PROCUREMENT' WHERE entity_type IN ('PurchaseOrder', 'GRN');
UPDATE workflow.ApprovalRules SET module_code = 'SERVICES' WHERE entity_type IN ('ServiceOrder');
UPDATE workflow.ApprovalRules SET module_code = 'LOGISTICS' WHERE entity_type IN ('DeliveryOrder', 'Shipments');
UPDATE workflow.ApprovalRules SET module_code = 'WAREHOUSE' WHERE entity_type IN ('StockTransfer', 'StockAdjustment');
UPDATE workflow.ApprovalRules SET module_code = 'INVENTORY' WHERE entity_type IN ('StockCount');
-- Rules with module_code = NULL apply regardless of module status
```

Behavior: The workflow engine checks `module_code` when evaluating rules. If the associated module is disabled, the rule is skipped (dormant). It fires again when re-enabled.

10.3 Permissions — Module Association

New column on `auth.Permissions`:

Column	Type	Constraints	Description
<code>module_code</code>	VARCHAR(30)	NULL	Module this permission belongs to

Backfill existing permissions:

```
sql
```

```
UPDATE auth.Permissions SET module_code = 'PRODUCTION' WHERE code LIKE 'production.%' OR code LIKE 'material';
UPDATE auth.Permissions SET module_code = 'PROCUREMENT' WHERE code LIKE 'procurement.%' OR code LIKE 'purchase';
UPDATE auth.Permissions SET module_code = 'SERVICES' WHERE code LIKE 'service.%';
UPDATE auth.Permissions SET module_code = 'WAREHOUSE' WHERE code LIKE 'warehouse.%';
UPDATE auth.Permissions SET module_code = 'LOGISTICS' WHERE code LIKE 'logistics.%' OR code LIKE 'delivery';
UPDATE auth.Permissions SET module_code = 'FINANCE' WHERE code LIKE 'finance.%' OR code LIKE 'journal.%';
UPDATE auth.Permissions SET module_code = 'INVENTORY' WHERE code LIKE 'inventory.%' OR code LIKE 'product.%';
```

Behavior in Role Management UI: When assigning permissions to a role, the permission list is grouped by module. Permissions for disabled modules are shown greyed out with a tooltip: "Enable the {Module} module to use this permission." They can still be assigned (pre-configuration) but won't be enforced until the module is enabled.

10.4 Sync Timestamps — `modified_at` on Shared Entities

Add `modified_at` column to the following existing tables:

Table	Purpose
<code>inventory.Products</code>	Already added in C5
<code>lookups.TaxRates</code>	POS needs tax delta sync
<code>lookups.UnitOfMeasures</code>	POS needs UoM delta sync
<code>lookups.Categories</code>	POS needs category delta sync
<code>inventory.BillOfMaterials</code>	Sync BOM changes to offline clients
<code>inventory.BomLines</code>	Sync BOM line changes
<code>customers.Customers</code>	Created with <code>modified_at</code> in C8
<code>inventory.Warehouses</code>	Sync warehouse reference data

SQL trigger template (applied to each table):

```
sql
CREATE OR ALTER TRIGGER trg_{Table}_ModifiedAt ON {schema}.{Table}
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE t SET t.modified_at = SYSUTCDATETIME()
    FROM {schema}.{Table} t
    INNER JOIN inserted i ON t.{pk} = i.{pk};
END;
```

Why triggers instead of EF Core interceptors? Triggers guarantee `modified_at` is updated even for direct SQL updates, bulk operations, and other applications that bypass EF Core. The EF Core `SaveChangesInterceptor` supplements this for in-application change tracking.

11. C10 — Frontend Module Context & Module Administration UI

11.1 Rationale

The React frontend needs a centralized way to check module and feature enablement, and tenant admins need a UI to manage their modules.

11.2 Frontend Components

11.2.1 `ModuleProvider` (React Context)

```
tsx

interface ModuleContextType {
  enabledModules: string[];           // ['CORE', 'INVENTORY', 'PROCUREMENT', ...]
  enabledFeatures: string[];         // ['PURCHASE_ORDERS', 'GRN_MANAGEMENT', ...]
  isModuleEnabled: (code: string) => boolean;
  isFeatureEnabled: (code: string) => boolean;
  isLoading: boolean;
  refresh: () => void;
}
```

- Fetches `/api/tenant/modules/enabled` on app mount
- Caches in React state — refreshed on module enable/disable or page reload
- Provides context to all child components

11.2.2 `ModuleGuard` Component

```
tsx

interface ModuleGuardProps {
  module?: string;           // Module code
  feature?: string;          // Feature code
  children: React.ReactNode;
  fallback?: React.ReactNode; // What to show when disabled (default: null/hidden)
}
```

Usage in sidebar:

```
tsx

<ModuleGuard module="PRODUCTION">
  <NavItem to="/production" icon="factory">Production</NavItem>
</ModuleGuard>
```

Usage in Product form:

```
tsx
```

```
<Tabs>
  <Tab label="Basic" />
  <Tab label="Inventory" />
  <ModuleGuard module="PROCUREMENT">
    <Tab label="Procurement" />
  </ModuleGuard>
  <ModuleGuard feature="BOM_MANAGEMENT">
    <Tab label="BOM" />
  </ModuleGuard>
  <ModuleGuard module="PRODUCTION">
    <Tab label="Production" />
  </ModuleGuard>
  <ModuleGuard module="SERVICES">
    <Tab label="Service" />
  </ModuleGuard>
  <Tab label="Finance" />
</Tabs>
```

11.2.3 useModule Hook

```
tsx

const { isEnabled, isFeatureEnabled } = useModule();

// In component logic
if (isEnabled('POS')) {
  // Show POS-related data
}
```

11.3 Module Administration UI

Route: /settings/modules

Access: Tenant Admin role only

11.3.1 Module List View

Section	Content
Header	"Module Management" title + tenant name
Core Modules	Always-on modules shown as cards with "Always Active" badge — no toggle
Active Modules	Enabled toggleable modules shown as cards with toggle switch
Available Modules	Disabled but available modules shown as greyed cards with "Enable" button
Coming Soon	Future modules (<code>is_available = 0</code>) shown as preview cards with "Coming Soon" badge

Each module card shows:

- Module icon + name
- Description (one line)
- Status badge: Active (green), Grace Period (amber with countdown), Disabled (grey), Coming Soon (blue)
- Dependencies: "Requires: Inventory, Suppliers"

- Feature count: "4 of 6 features enabled"
- Toggle switch (for toggleable modules)
- "Manage Features" expand/chevron

11.3.2 Module Detail / Feature Management

Clicking "Manage Features" expands to show:

Column	Description
Feature name	Display name
Status	Toggle switch (core features show as locked-on)
Description	One-line description
Availability	"Active" or "Coming Soon"
Required By	Other features that depend on this one

11.3.3 Disable Module Confirmation Dialog

Triggered when toggling a module off:

Disable Production Module?

This will:

- Block new Production Order creation
- Hide Production menu and Product form tabs
- Stop MRP scheduling background jobs

This will NOT:

- Delete any existing data
- Remove historical ledger entries
- Affect BOM definitions (shared with Services)

In-progress records:

3

 3 Production Orders are currently in progress.

They will have a 30-day grace period to complete.

Grace Period: [30] days

[Cancel]

[Disable Module]

Dependency validation — Blocking dialog if dependents are active:

```
|| Cannot Disable Inventory ||
||
|| The following active modules depend on Inventory: ||
|| • Production ||
|| • Services ||
|| • Procurement ||
||
|| Disable those modules first. ||
||
|| [OK] ||
```

12. Database Migrations

Migration	Description	Schema	Depends On
M037.1	Create <code>tenant.Modules</code> table + seed catalog data	tenant	—
M037.2	Create <code>tenant.ModuleDependencies</code> table + seed dependency data	tenant	M037.1
M037.3	Create <code>tenant.ModuleFeatures</code> table + seed feature data	tenant	M037.1
M037.4	Create <code>tenant.OrgModules</code> table + seed all existing tenants	tenant	M037.1
M037.5	Create <code>tenant.OrgModuleFeatures</code> table + seed	tenant	M037.3, M037.4
M037.6	Create <code>tenant.OrgModuleHistory</code> table	tenant	M037.4
M037.7	Add capability flags + <code>modified_at</code> to <code>inventory.Products</code>	inventory	—
M037.8	Create <code>material.ProductProductionSettings</code> + <code>service.ProductServiceSettings</code> extension tables	material, service	M037.7
M037.9	Transfer <code>BillOfMaterials</code> + <code>BomLines</code> from <code>material</code> to <code>inventory</code> schema + add <code>bom_usage</code> column	inventory, material	M037.7
M037.10	Create <code>inventory.Routes</code> + <code>inventory.ProductRoutes</code> + seed routes	inventory	M037.4
M037.11	Create <code>customers</code> schema + <code>customers.Customers</code> table + walk-in seed	customers	—
M037.12	Add <code>module_code</code> to <code>workflow.ApprovalRules</code> + backfill	workflow	M037.1
M037.13	Add <code>module_code</code> to <code>auth.Permissions</code> + backfill	auth	M037.1
M037.14	Add <code>modified_at</code> + triggers to shared lookup tables (<code>TaxRates</code> , <code>UnitOfMeasures</code> , <code>Categories</code> , <code>Warehouses</code>)	lookups, inventory	—

Migration SQL — Key Migrations

M037.1 — Module Catalog

sql

```

CREATE TABLE tenant.Modules (
  module_id      INT IDENTITY(1,1) PRIMARY KEY,
  code           VARCHAR(30)      NOT NULL,
  name           NVARCHAR(100)   NOT NULL,
  description     NVARCHAR(500)   NULL,
  schema_names   VARCHAR(200)    NOT NULL,
  is_always_on   BIT             NOT NULL DEFAULT 0,
  is_available   BIT             NOT NULL DEFAULT 1,
  icon           VARCHAR(50)      NULL,
  sort_order     INT             NOT NULL DEFAULT 0,
  created_at     DATETIME2(2)     NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT UQ_Modules_Code UNIQUE (code)
);

-- Seed (see §2.3 for full list)
INSERT INTO tenant.Modules (code, name, schema_names, is_always_on, is_available, sort_order) VALUES
  ('CORE',      'Core Platform',      'auth,tenant,lookups,workflow,hangfire,reports', 1, 1, 1),
  ('INVENTORY', 'Inventory Management', 'inventory', 1, 1, 2),
  ('FINANCE',   'Finance & Accounting', 'finance', 1, 1, 3),
  ('CUSTOMERS', 'Customer Management',  'customers', 1, 1, 4),
  ('SUPPLIERS', 'Supplier Management',   'suppliers', 0, 1, 5),
  ('PROCUREMENT', 'Procurement',       'procurement', 0, 1, 6),
  ('DEMAND',    'Demand Planning',     'demand', 0, 1, 7),
  ('WAREHOUSE', 'Warehouse Operations', 'warehouse', 0, 1, 8),
  ('LOGISTICS', 'Logistics & Shipping', 'logistics', 0, 1, 9),
  ('PRODUCTION', 'Manufacturing & Production', 'material', 0, 1, 10),
  ('SERVICES',  'Service Management',   'service', 0, 1, 11),
  ('POS',       'Point of Sale',        'pos', 0, 0, 12),
  ('PROJECTS',  'Project Management',   'projects', 0, 0, 13),
  ('CRM',       'Customer Relationship', 'crm', 0, 0, 14),
  ('MAINTENANCE', 'Maintenance / CMMS', 'maintenance', 0, 0, 15),
  ('ECOMMERCE', 'E-Commerce Integration', 'ecommerce', 0, 0, 16)

```

M037.9 — BOM Schema Transfer

sql

```

-- Transfer BOM tables from material to inventory schema
ALTER SCHEMA inventory TRANSFER material.BillOfMaterials;
ALTER SCHEMA inventory TRANSFER material.BomLines;

-- Add bom_usage classification column
ALTER TABLE inventory.BillOfMaterials
    ADD bom_usage TINYINT NOT NULL DEFAULT 0;
    -- 0=Universal, 1=ProductionPreferred, 2=ServicePreferred, 3=PosKit

-- Add modified_at for sync support
ALTER TABLE inventory.BillOfMaterials
    ADD modified_at DATETIME2(2) NOT NULL DEFAULT SYSUTCDATETIME();

ALTER TABLE inventory.BomLines
    ADD modified_at DATETIME2(2) NOT NULL DEFAULT SYSUTCDATETIME();

-- Triggers for modified_at
CREATE OR ALTER TRIGGER trg_BillOfMaterials_ModifiedAt ON inventory.BillOfMaterials
AFTER UPDATE AS
BEGIN
    SET NOCOUNT ON;
    UPDATE inventory.BillOfMaterials SET modified_at = SYSUTCDATETIME()
    WHERE bom_id IN (SELECT bom_id FROM inserted);
END;

CREATE OR ALTER TRIGGER trg_BomLines_ModifiedAt ON inventory.BomLines
AFTER UPDATE AS
BEGIN
    SET NOCOUNT ON;
    UPDATE inventory.BomLines SET modified_at = SYSUTCDATETIME()
    WHERE bom_line_id IN (SELECT bom_line_id FROM inserted);
END;

```

M037.11 — Customer Schema

```
sql
```

```
CREATE SCHEMA customers;
```

```
CREATE TABLE customers.Customers (  
    customer_id          BIGINT IDENTITY(1,1) PRIMARY KEY,  
    org_id               BIGINT          NOT NULL,  
    code                 VARCHAR(20)     NOT NULL,  
    name                 NVARCHAR(200)   NOT NULL,  
    customer_type        TINYINT         NOT NULL DEFAULT 1,  
    email                NVARCHAR(200)   NULL,  
    phone                VARCHAR(30)     NULL,  
    mobile                VARCHAR(30)     NULL,  
    tax_id               VARCHAR(50)     NULL,  
    credit_limit          DECIMAL(18,4)  NOT NULL DEFAULT 0,  
    current_balance       DECIMAL(18,4)  NOT NULL DEFAULT 0,  
    loyalty_points        INT            NOT NULL DEFAULT 0,  
    payment_terms_days    INT            NOT NULL DEFAULT 0,  
    currency_code         VARCHAR(3)     NOT NULL DEFAULT 'USD',  
    address_line1         NVARCHAR(200)   NULL,  
    address_line2         NVARCHAR(200)   NULL,  
    city                 NVARCHAR(100)   NULL,  
    state_province        NVARCHAR(100)   NULL,  
    postal_code           VARCHAR(20)     NULL,  
    country_code          VARCHAR(3)     NULL,  
    notes                 NVARCHAR(MAX)   NULL,  
    is_active             BIT            NOT NULL DEFAULT 1,  
    created_at            DATETIME2(2)   NOT NULL DEFAULT SYSUTCDATETIME(),  
    created_by            BIGINT          NOT NULL,  
    modified_at           DATETIME2(2)   NOT NULL DEFAULT SYSUTCDATETIME(),  
    modified_by           BIGINT          NULL,  
    row_version           ROWVERSION     NOT NULL,  
    CONSTRAINT UQ_Customers_OrgCode UNIQUE (org_id, code),  
    CONSTRAINT FK_Customers_Org FOREIGN KEY (org_id) REFERENCES tenant.Organizations(org_id)  
);
```

```
CREATE INDEX IX_Customers_OrgName ON customers.Customers (org_id, name);  
CREATE INDEX IX_Customers_OrgEmail ON customers.Customers (org_id, email) WHERE email IS NOT NULL;  
CREATE INDEX IX_Customers_OrgPhone ON customers.Customers (org_id, phone) WHERE phone IS NOT NULL;  
CREATE INDEX IX_Customers_OrgType ON customers.Customers (org_id, customer_type);  
CREATE INDEX IX_Customers_OrgModified ON customers.Customers (org_id, modified_at);
```

```
-- Trigger for modified_at
```

```
CREATE OR ALTER TRIGGER trg_Customers_ModifiedAt ON customers.Customers  
AFTER UPDATE AS  
BEGIN  
    SET NOCOUNT ON;  
    UPDATE customers.Customers SET modified_at = SYSUTCDATETIME()  
    WHERE customer_id IN (SELECT customer_id FROM inserted);  
END;
```

```
-- Seed walk-in customer per org
```

```
INSERT INTO customers.Customers (org_id, code, name, customer_type, created_by)  
SELECT o.org_id, 'C-WALKIN', 'Walk-In Customer', 0,
```

```
(SELECT TOP 1 user_id FROM auth.Users u WHERE u.org_id = o.org_id ORDER BY user_id)
FROM tenant.Organizations o;
```

13. API Endpoints

13.1 Module Management API

#	Method	Endpoint	Auth	Description
1	GET	<code>/api/tenant/modules</code>	Admin	List all modules in catalog with dependency and feature info
2	GET	<code>/api/tenant/modules/enabled</code>	Any	List enabled module + feature codes for current tenant (used by frontend ModuleProvider)
3	POST	<code>/api/tenant/modules/{code}/enable</code>	Admin	Enable a module (validates dependencies)
4	POST	<code>/api/tenant/modules/{code}/disable</code>	Admin	Disable a module (validates dependents, sets grace period)
5	PUT	<code>/api/tenant/modules/{code}/features/{featureCode}</code>	Admin	Enable/disable a feature within a module
6	GET	<code>/api/tenant/modules/{code}/history</code>	Admin	Get enable/disable history for a module
7	GET	<code>/api/tenant/modules/{code}/impact</code>	Admin	Pre-disable impact check: in-progress records, dependent modules

13.2 Customer API

#	Method	Endpoint	Auth	Description
8	GET	<code>/api/customers</code>	customers.view	List customers (paginated, filterable)
9	GET	<code>/api/customers/{id}</code>	customers.view	Get customer detail
10	POST	<code>/api/customers</code>	customers.create	Create customer
11	PUT	<code>/api/customers/{id}</code>	customers.update	Update customer
12	PATCH	<code>/api/customers/{id}/status</code>	customers.update	Activate/deactivate
13	GET	<code>/api/customers/search?q={term}</code>	customers.view	Quick search for lookups

13.3 BOM API (Relocated)

#	Method	Endpoint	Auth	Notes
14	GET	/api/bom	inventory.bom.view	Relocated from /api/production/bom
15	GET	/api/bom/{id}	inventory.bom.view	Relocated
16	POST	/api/bom	inventory.bom.create	Relocated
17	PUT	/api/bom/{id}	inventory.bom.update	Relocated
18	DELETE	/api/bom/{id}	inventory.bom.delete	Relocated

Backward compatibility: The old /api/production/bom/* routes redirect (301) to /api/bom/* for one release cycle.

13.4 Routes API

#	Method	Endpoint	Auth	Description
19	GET	/api/routes	inventory.view	List available routes for current tenant
20	GET	/api/products/{id}/routes	inventory.view	Get available routes for a product (filtered by module enablement)
21	PUT	/api/products/{id}/routes	inventory.update	Set product routes (array of route_ids + preferred flag)

13.5 Sync API (Foundation for POS)

#	Method	Endpoint	Auth	Description
22	GET	/api/sync/catalog?since={iso8601}	Any	Delta sync: products, categories, taxes, UoMs changed since timestamp
23	GET	/api/sync/customers?since={iso8601}	customers.view	Delta sync: customers changed since timestamp

14. Permission Claims

14.1 New Permissions

Code	Module	Description
tenant.modules.view	CORE	View module configuration
tenant.modules.manage	CORE	Enable/disable modules and features
customers.view	CUSTOMERS	View customer records
customers.create	CUSTOMERS	Create new customers
customers.update	CUSTOMERS	Update customer records
customers.delete	CUSTOMERS	Deactivate customers
inventory.bom.view	INVENTORY	View BOMs (relocated from production)
inventory.bom.create	INVENTORY	Create BOMs
inventory.bom.update	INVENTORY	Update BOMs
inventory.bom.delete	INVENTORY	Delete BOMs
inventory.routes.view	INVENTORY	View fulfillment routes
inventory.routes.manage	INVENTORY	Configure product routes

15. UI Wireframes & Specifications

15.1 Module Administration Page

Route: /settings/modules

Layout: Grid of module cards, grouped by classification.

Section 1 — Core Modules:

 Core

Always Active

6 features

[Manage ▼]

 Inventory

Always Active

5/7 features

[Manage ▼]

 Finance

Always Active

2/5 features

[Manage ▼]

 Customers

Always Active

1/3 features

[Manage ▼]

Section 2 — Active Modules:

 Production

Active

4/6 features

[Manage ▼]

 Procurement

Active

3/5 features

[Manage ▼]

 Warehouse

Active

3/5 features

[Manage ▼]

Section 3 — Coming Soon:

 Point of Sale

Coming Soon

 Projects

Coming Soon

 CRM

Coming Soon

15.2 Product Form — Conditional Tabs

Tab visibility rules:

Tab	Condition
Basic	Always
Inventory	Always
Procurement	PROCUREMENT module enabled
BOM	BOM_MANAGEMENT feature enabled (auto-enabled when PRODUCTION or SERVICES active)
Production	PRODUCTION module enabled AND product <code>is_manufacturable = true</code>
Service	SERVICES module enabled AND product <code>is_serviceable = true</code>
POS	POS module enabled AND product <code>is_sellable = true</code> (future)
Finance	Always

15.3 Customer List Page

Route: `/customers`

Standard SMS list page pattern:

Column	Width	Sortable	Filterable
Code	100px	✓	✓
Name	250px	✓	✓ (search)
Type	100px	✓	✓ (dropdown)
Phone	130px	✗	✓ (search)
Email	200px	✗	✓ (search)
Credit Limit	120px	✓	✗
Balance	120px	✓	✗
Status	80px	✓	✓

Actions: + New Customer, Export

15.4 Customer Detail Page

Route: `/customers/{id}`

Tabs:

Tab	Content
Details	Name, type, contact info, address, tax ID, payment terms, notes
Financial	Credit limit, current balance, payment history (future), outstanding invoices (future)
Orders	Sale orders for this customer (future — linked when Sales module exists)
Timeline	Document timeline entries

15.5 Sidebar Navigation — Module-Aware

Navigation structure with ModuleGuard:

16. Business Rules

16.1 Module Lifecycle Rules

Rule ID	Rule	Description
MOD-01	Always-on modules cannot be disabled	Modules with <code>is_always_on = 1</code> (CORE, INVENTORY, FINANCE, CUSTOMERS) reject disable requests
MOD-02	Dependency validation on enable	Enabling a module first checks all dependencies are enabled. If PROCUREMENT depends on INVENTORY + SUPPLIERS, both must be enabled before PROCUREMENT can be enabled
MOD-03	Dependent validation on disable	Disabling a module checks for active dependents. Cannot disable INVENTORY if PRODUCTION is still enabled. Returns list of dependents that must be disabled first
MOD-04	Grace period on disable	When a module is disabled, <code>grace_period_ends_at</code> is set to <code>disabled_at + {graceDays}</code> . During grace period, existing records can be completed/closed/cancelled but no new records created
MOD-05	Grace period expiry	A Hangfire daily job checks for expired grace periods and updates <code>grace_period_ends_at = NULL</code> + logs to history with action = GracePeriodExpired
MOD-06	Feature requires parent module	A feature can only be enabled if its parent module is enabled. Disabling a module auto-disables all its features
MOD-07	Core features always on	Features with <code>is_core_feature = 1</code> cannot be individually disabled — they are always on when the parent module is enabled
MOD-08	BOM_MANAGEMENT auto-enable	When PRODUCTION or SERVICES module is enabled, the BOM_MANAGEMENT feature in INVENTORY is automatically enabled. When both are disabled, system-auto-enabled BOM_MANAGEMENT is auto-disabled (manually-enabled by admin is preserved)
MOD-09	Module action audit	Every enable/disable/feature-toggle action is logged in <code>OrgModuleHistory</code> with performing user, timestamp, and action type
MOD-10	Re-enable restores access	Re-enabling a previously disabled module restores full access to all existing data. No data migration required
MOD-11	Unavailable modules blocked	Modules with <code>is_available = 0</code> cannot be enabled — they are placeholder catalog entries for future modules

16.2 Product Capability Rules

Rule ID	Rule	Description
PRD-CAP-01	Capability flags independent of module	<code>is_sellable</code> , <code>is_manufacturable</code> , etc. are product attributes set by the user. Module toggle does not change flag values
PRD-CAP-02	Tab visibility = module + flag	Product form tab visibility requires BOTH the module enabled AND the relevant flag set to true (see §15.2)
PRD-CAP-03	Extension data conditional loading	API response includes module extension data only when the module is enabled. When disabled, extension key is absent (not null)
PRD-CAP-04	Extension data preserved	Disabling a module does not delete extension table records. Re-enabling restores them in API responses

16.3 Route Rules

Rule ID	Rule	Description
RTE-01	Route filtered by module	Available routes for a product are dynamically filtered by module enablement at query time
RTE-02	Preferred route fallback	If a product's preferred route becomes unavailable (its module disabled), the next available route by sequence becomes the effective preferred
RTE-03	No available routes warning	If all of a product's routes become unavailable, the product can still be purchased/sold but fulfillment routing shows a warning

16.4 BOM Sharing Rules

Rule ID	Rule	Description
BOM-SHR-01	BOM is module-agnostic	BOM data in <code>inventory</code> schema is accessible regardless of which production/service modules are enabled
BOM-SHR-02	BOM_MANAGEMENT controls create/edit	Creating or editing BOMs requires the BOM_MANAGEMENT feature to be enabled
BOM-SHR-03	BOM read access always available	Reading BOM data (for reports, historical reference) is always available through INVENTORY module
BOM-SHR-04	BOM usage is advisory	The <code>bom_usage</code> field is a filter/sort preference, not an access control gate
BOM-SHR-05	BOM explosion per module	Production explosion creates ProductionMaterialRequirements (material schema). Service explosion creates ServiceMaterialRequirements (service schema). Each module's explosion service references the shared BOM

16.5 Customer Rules

Rule ID	Rule	Description
CUST-01	Walk-in customer immutable	The system-seeded Walk-In Customer (code = C-WALKIN) cannot be deleted or deactivated
CUST-02	Customer code auto-generation	Format: C-{5-digit sequence per org}. e.g., C-00001, C-00002
CUST-03	Credit limit enforcement	Future: POS and Sales modules check <code>current_balance + new_order_total <= credit_limit</code> for credit sales
CUST-04	Customer type validation	WalkIn (0) customers cannot have credit_limit > 0

17. Test Scenarios

ID	Scenario	Expected Result
TS-037-01	Enable PRODUCTION module when INVENTORY is enabled	Success — PRODUCTION enabled, BOM_MANAGEMENT auto-enabled, history logged
TS-037-02	Enable PROCUREMENT when SUPPLIERS is disabled	Fail — "Enable Suppliers module first"
TS-037-03	Disable INVENTORY when PRODUCTION is active	Fail — "Disable Production first"
TS-037-04	Disable PRODUCTION with 3 in-progress Production Orders	Success — module disabled, grace_period_ends_at set to +30 days, history logged
TS-037-05	Create new Production Order during grace period	Fail — 403 MODULE_NOT_LICENSED
TS-037-06	Complete existing Production Order during grace period	Success — state transition allowed
TS-037-07	Access Production Order after grace period expired	GET succeeds (AllowReadWhenDisabled), POST/PUT fail with 403
TS-037-08	Re-enable PRODUCTION after disable	Success — all data accessible, in-progress orders resume, grace cleared
TS-037-09	Disable PRODUCTION, verify BOM still accessible	BOM API (/api/bom) returns 200, BOM tab visible (if SERVICES active)
TS-037-10	Disable both PRODUCTION and SERVICES	BOM_MANAGEMENT auto-disabled, BOM tab hidden, BOM data preserved
TS-037-11	Enable SERVICES only (no PRODUCTION), create Service Order with BOM	Success — BOM accessible, explosion works, Service Order created
TS-037-12	Toggle MRP_SCHEDULING feature off within PRODUCTION	MRP job skips tenant, Production Orders still work, MRP menu hidden
TS-037-13	Product form with all modules enabled	All tabs visible: Basic, Inventory, Procurement, BOM, Production, Service, Finance
TS-037-14	Product form with only INVENTORY + FINANCE	Only tabs visible: Basic, Inventory, Finance
TS-037-15	Product with is_manufacturable=true, PRODUCTION disabled	Production tab hidden, flag preserved, re-enabling shows tab again

ID	Scenario	Expected Result
TS-037-16	GET /api/products/{id} with PRODUCTION enabled	Response includes <code>productionSettings</code> object
TS-037-17	GET /api/products/{id} with PRODUCTION disabled	Response has no <code>productionSettings</code> key
TS-037-18	Product routes: BUY + MANUFACTURE configured, PRODUCTION disabled	GET /api/products/{id}/routes returns only BUY
TS-037-19	Product routes: preferred is MANUFACTURE, PRODUCTION disabled	Effective preferred falls back to next by sequence (BUY)
TS-037-20	Create customer with type WalkIn and credit_limit > 0	Fail — validation error
TS-037-21	Delete system Walk-In customer	Fail — protected record
TS-037-22	Customer search by phone number	Returns matching customers, ordered by relevance
TS-037-23	Delta sync /api/sync/catalog?since= returns only modified products	Only products with modified_at > since timestamp returned
TS-037-24	Sidebar navigation with PRODUCTION + SERVICES disabled	Production and Services menu groups hidden, rest visible
TS-037-25	Module admin: disable module shows impact dialog with in-progress count	Dialog shows correct count of in-progress records
TS-037-26	Workflow rule with module_code = PRODUCTION, module disabled	Approval rule dormant — never evaluates
TS-037-27	Permission with module_code = PRODUCTION, module disabled	Permission greyed in role management, not enforced
TS-037-28	Enable unavailable module (is_available = 0)	Fail — "Module not yet available"
TS-037-29	Material Issue for Service Order when PRODUCTION disabled	Success — Material Issue checks SERVICES module, not PRODUCTION
TS-037-30	Concurrent module enable/disable from two admins	OrgModules row_version prevents conflict
TS-037-31	BOM with bom_usage = ProductionPreferred, listed in Service Order BOM picker	Appears in list (usage is advisory, not restrictive)
TS-037-32	Hangfire MRP job runs, tenant has PRODUCTION disabled	Job skips tenant, no error logged
TS-037-33	Grace period daily job expires a module's grace	OrgModules.grace_period_ends_at set to NULL, history logged as GracePeriodExpired

ID	Scenario	Expected Result
TS-037-34	Full flow: enable fresh tenant with Trading Edition (INVENTORY + PROCUREMENT + SUPPLIERS + FINANCE + CUSTOMERS)	All enabled, sidebar shows only those modules, product form shows relevant tabs
TS-037-35	Full flow: upgrade Trading tenant to Manufacturing (add PRODUCTION + WAREHOUSE)	Both modules enabled, BOM_MANAGEMENT auto-enabled, sidebar expands, product form adds tabs

18. Development Phases

18.1 Phase Overview

Phase	Focus	Days	Tasks
Phase 1	Module Catalog & Licensing Tables	4	M037.1–M037.6 + seed data
Phase 2	Module Access Infrastructure	5	IModuleService + middleware + attributes + controller tagging + job retrofit
Phase 3	Product Extensions & BOM Migration	5	M037.7–M037.9 + extension tables + BOM transfer + EF Core changes
Phase 4	Routes + Customer Master	4	M037.10–M037.11 + Route/Customer entities + API + UI
Phase 5	Cross-Module Awareness & Frontend	5	M037.12–M037.14 + workflow/permission backfill + React ModuleProvider + Admin UI
Total		23 days	14 migrations, 8 new tables, 6 modified tables

18.2 Phase Details

Phase 1 — Module Catalog & Licensing (4 days)

Track	Tasks	Days
1A	M037.1 + M037.2 — Modules + Dependencies tables, seed data	1.5
1B	M037.3 — ModuleFeatures table, seed all features	1
1C	M037.4 + M037.5 + M037.6 — OrgModules, OrgModuleFeatures, OrgModuleHistory + tenant seed	1.5

Phase 2 — Module Access Infrastructure (5 days)

Track	Tasks	Days
2A	IModuleService implementation with caching + DI registration	1.5
2B	RequiresModuleAttribute + RequiresFeatureAttribute + ModuleAccessMiddleware	1.5
2C	Tag all existing controllers with [RequiresModule] (~30 controllers)	1
2D	ModuleAwareJob base class + retrofit existing Hangfire jobs (~8 jobs)	1

Phase 3 — Product Extensions & BOM Migration (5 days)

Track	Tasks	Days
3A	M037.7 — Product capability flags + modified_at on Products	1
3B	M037.8 — ProductProductionSettings + ProductServiceSettings extension tables	1
3C	M037.9 — BOM schema transfer (material → inventory) + bom_usage + triggers	1.5
3D	EF Core DbContext reconfiguration + BomService DI relocation + API route redirect	1.5

Phase 4 — Routes + Customer Master (4 days)

Track	Tasks	Days
4A	M037.10 — Routes + ProductRoutes tables + seed data + available routes query service	1.5
4B	M037.11 — Customer schema + Customers table + walk-in seed + Customer API	1.5
4C	Customer List + Detail UI pages	1

Phase 5 — Cross-Module Awareness & Frontend (5 days)

Track	Tasks	Days
5A	M037.12 + M037.13 — workflow/permission module_code + backfill	1
5B	M037.14 — modified_at columns + triggers on lookup tables	0.5
5C	React ModuleProvider + ModuleGuard + useModule hook	1
5D	Sidebar module-aware navigation + Product form conditional tabs	1
5E	Module Administration UI (module cards, feature toggles, disable confirmation dialog)	1.5

18.3 Dependencies



19. Future Enhancements

The following are **not included** in this addendum and will be addressed in their respective module addendums:

#	Feature	Target Addendum	Description
1	POS Module	Addendum 38+	Point of Sale — terminals, sessions, orders, payments, PWA, offline sync
2	POS Product Extension	Addendum 38+	<code>pos.ProductPosSettings</code> — barcode, POS display name, sale price, POS category
3	Sales Module	Future	Sales Orders, Quotations, Sales Pipeline
4	Project Module	Future	Project Orders, WBS, project-based material consumption
5	CRM Module	Future	Leads, Opportunities, Customer relationship tracking
6	Maintenance / CMMS	Future	Preventive/corrective maintenance, asset-based scheduling
7	E-Commerce Integration	Future	Product catalog API, stock availability, order ingestion
8	SignalR Real-Time Hub	POS Addendum	Real-time stock updates, price change push for POS terminals
9	PWA Infrastructure	POS Addendum	Service Worker, IndexedDB (Dexie), offline-first client
10	License Tier Enforcement	Future	Commercial packaging: Starter/Professional/Enterprise editions with module bundles
11	Customer Portal	Future	Self-service customer portal for orders and service requests
12	Multi-Company / Intercompany	Future	Multiple legal entities within one tenant, intercompany transfers

Appendix A — Entity Relationship Summary



Appendix B — Module Edition Packaging (Reference)

This is a commercial reference — not enforced by this addendum. License tier enforcement is deferred to a future addendum.

Edition	Included Modules	Target Market
Trading	CORE + INVENTORY + FINANCE + CUSTOMERS + SUPPLIERS + PROCUREMENT	Import/export, wholesale, distribution
Manufacturing	Trading + PRODUCTION + WAREHOUSE + DEMAND	Factories, assembly, make-to-order
Services	CORE + INVENTORY + FINANCE + CUSTOMERS + SERVICES	Field service, maintenance, consulting
Retail	Trading + POS + WAREHOUSE	Shops, retail chains
Enterprise	All modules	Large organizations needing everything

Add-on modules (available with any edition): LOGISTICS, DEMAND, individual feature upgrades within modules.